

Striver SDE Sheet - Greedy Revision Notes

Short DSA revision notes for Greedy problems

Use this as a quick last-minute revision sheet. Focus on the pattern, decision rule, and complexity.

Core Patterns

Pattern	Used In
Sort by ending time	N Meetings, Activity Selection
Sort + two pointers	Minimum Platforms
Sort by profit/value	Job Sequencing, Fractional Knapsack
Largest valid choice first	Minimum Number of Coins

Problem-wise Notes

1. N Meetings in One Room

Problem	Select the maximum number of non-overlapping meetings.
Approach	Sort meetings by ending time. Always select the meeting that ends earliest and does not overlap with the last selected meeting.
Steps	1. Store start, end, and original position. 2. Sort meetings by end time. 3. Pick the first meeting. 4. For each next meeting, pick it only if start > lastEnd.
Key Idea	The meeting that finishes early leaves maximum space for future meetings.
Complexity	Time: $O(n \log n)$, Space: $O(n)$

2. Minimum Number of Platforms

Problem	Find minimum platforms needed so no train waits.
Approach	Sort arrival and departure times separately. Use two pointers to count how many trains are currently at the station.
Steps	1. Sort arrivals and departures. 2. If arrival $\leq$ departure, one more platform is needed. 3. Else, one train leaves and one platform becomes free. 4. Track the maximum platform count.
Key Idea	The answer is the maximum number of trains present at the station at the same time.
Complexity	Time: $O(n \log n)$, Space: $O(1)$

3. Job Sequencing Problem

Problem	Schedule jobs before deadlines to maximize profit.
Approach	Sort jobs by profit in descending order. Place every job in the latest available slot before its deadline.
Steps	1. Sort jobs by profit high to low. 2. Create time slots up to max deadline. 3. For every job, check backward from deadline. 4. Put the job in the first empty slot.
Key Idea	Do the most profitable job first, but place it as late as possible.
Complexity	Time: $O(n \log n + n * \text{maxDeadline})$, Space: $O(\text{maxDeadline})$

4. Fractional Knapsack

Problem	Maximize value in a knapsack when items can be broken into fractions.
Approach	Sort items by value/weight ratio in descending order. Take the highest ratio items first.
Steps	1. Calculate value/weight ratio. 2. Sort items by ratio descending. 3. Take full item if capacity allows.

	4. Otherwise, take the required fraction and stop.
Key Idea	Choose the item giving maximum value per unit weight.
Complexity	Time: $O(n \log n)$, Space: $O(1)$

5. Minimum Number of Coins

Problem	Find the minimum number of coins needed to make a value using standard denominations.
Approach	Always pick the largest coin possible, then reduce the amount.
Steps	1. Start from the largest denomination. 2. Take it while it is $\leq$ remaining amount. 3. Subtract its value from amount. 4. Move to the next smaller coin.
Key Idea	Greedy works for standard canonical coin systems such as Indian currency.
Complexity	Time: $O(\text{number of denominations})$, Space: $O(1)$

6. Activity Selection

Problem	Select maximum non-overlapping activities.
Approach	Sort activities by finish time and keep choosing activities that start after the last selected activity ends.
Steps	1. Sort activities by end time. 2. Pick the first activity. 3. Pick next activity only if start $>$ previous end. 4. Count all selected activities.
Key Idea	Same pattern as N Meetings: finish early to get more future choices.
Complexity	Time: $O(n \log n)$, Space: $O(1)$ or $O(n)$

Final Memory Line

Greedy means: sort/choose the best current option that keeps future choices open.